UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Tehnologia Informației

PROIECT DE DIPLOMĂ

Coordonator științific:
Ș.l.dr.ing. Cătălin Brăescu (florin-catalin.braescu@academic.tuiasi.ro)

Absolvent:
Mădălin Gherguț

Iași, 2025

UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Tehnologia Informației

Proiectarea și implementarea unei platforme web pentru managementul fluxurilor studențești utilizând Angular și PHP

PROIECT DE DIPLOMĂ

Coordonator științific:
Ș.l.dr.ing. Cătălin Brăescu (florin-catalin.braescu@academic.tuiasi.ro)

Absolvent:
Mădălin Gherguț

Iași, 2025

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII PROIECTULUI DE DIPLOMĂ

Subsemnatul _____,
legitimat cu ____ seria _____ nr. _____ , CNP_____
autorul lucrării _____
_____
_____
elaborată în vederea susținerii examenului de finalizare a studiilor de licență, programul de studii _____ organizat de către Facultatea de Automatică şi Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi" din Iaşi, sesiunea _____ a anului universitar _____, luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice „Gheorghe Asachi" din Iaşi (Manualul Procedurilor, UTI.POM.02 - Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) şi a convențiilor internaționale privind drepturile de autor.

Data Semnătura

Cuprins

Motivație **1**

Introducere **3**

Bibliografie **5**

Motivație

Procesul care a dus la dezvoltarea acestei aplicații nu a început dintr-o intenție pur tehnică, ci dintr-o observație persistentă a problemelor întâlnite în practica de zi cu zi a mediului universitar. Pe parcursul anilor de facultate, m-am angajat activ în diverse activități didactice și administrative, ceea ce mi-a oferit perspectiva atât a unui student, cât și a unui observator al proceselor instituționale. În acest context, platforma Moodle a fost instrumental importantă: a servit ca spațiu central pentru distribuirea materialelor de curs, comunicarea temelor și sarcinilor de lucru, precum și pentru urmărirea activităților și progresului didactic. Moodle rămâne, fără îndoială, o soluție utilă și solid construită pentru domeniul e-learning, cu o comunitate de dezvoltare activă și o bază de utilizatori impresionantă la nivel mondial.

Totuși, în interacțiunea de sezon cu sezon cu această platformă și cu procesele academice din jur, am observat că, deși Moodle acoperă bine aspectele de curs tradițional, în practica de zi cu zi a activităților de laborator și a administrării operaționale, situațiile reale ajung să genereze fragmente necontrolate de flux. Unele informații critice rămân înscrise în platformă, altele se migrează treptat în tabele locale gestionate de cadre didactice, în fișiere Excel care circulă prin email sau sunt stocate pe calculatoare personale, iar altele încă se evidențiază manual în caiete de prezenți. Această dispersie nu este o eroare a vreunui sistem individual, ci mai degrabă o spărtură sistematică între ceea ce o platformă generalist poate oferi și ceea ce practicienii universității au nevoie în momentul concret.

Această fragmentare generează o fricțiune administrativă care nu este nici spectaculoasă, nici catastrofală, dar nici neglijabilă. O sarcină care ar putea să fie simplă—actualizarea unei liste de prezență pe o sesiune de laborator, sau centralizarea elevilor unui grup pentru o activitate nouă—ajunge să implice mai mulți pași manuali, verificări repetate și sincronizări manuale între surse disparate. Pe măsură ce volumul de date și de tranzacții crește pe parcursul semestrului, costul acumulat al acestor operații deveni evident: apar întârzieri în procesarea informațiilor, apar diferențe semnificative între versiunile de date stocate în locuri diferite, iar luarea de decizii devine mai laborioasă din cauza lipsei unei imagini unice și actualizate asupra situației reale din teren. Din perspectiva utilizatorilor finali, experiența se fragmentează și devine inconsistentă: un student trebuie să caute informații în mai multe locuri, un profesor se află în situația de a repeta aceeași operație în mai multe sisteme în paralel, iar un administrator dedică o proporție disproporționată a timpului său la gestionarea cazurilor speciale și excepțiilor, în loc să poată construi și îmbunătăți procese standardizate și scalabile.

Motivația principală a acestei lucrări a fost, deci, construirea unei soluții specific adaptate acestui context local și acestor necesități reale, o aplicație care să completeze ceea ce lipsește din practica curentă și care să reducă substanțial dependența de fluxuri paralele, neintegrate și manual sincronizate. Ideea de bază nu a fost niciodată aceea de a înlocui o platformă consacrată și bine stabilită, ci mai degrabă de a proiecta o aplicație verticalizată, focalizată exclusiv pe managementul fluxurilor studențești în sens operațional, în particular în acele zone în care viteza de operare, claritatea și coerența datelor, și trasabilitatea auditabilă sunt esențiale pentru buna desfășurare a activităților.

Academo a fost conceput din start ca o aplicație web în care operațiile frecvente, repetitive, ar deveni naturale și intuitive de executat: autentificarea utilizatorilor cu diferențiere clară de roluri, administrarea și organizarea studenților în grupe funcționale, organizarea și programarea cursurilor și sesiunilor de laborator, marcarea și urmărirea prezenței în mod fluid și direct la locul de activitate, import expeditiv de date din fișiere Excel și acces rapid la rapoarte și statistici. Din perspective a experienței utilizatorului, obiectivul central a fost ca interfața să fie previzibilă și coerentă în toți pașii, suficient de flexibilă încât să se adapteze la ritmul și la particularitățile reale ale activităților din cadrul facultății, dar și suficient de structurată ca să imună utilizatorii de erori și să îi ghideze spre acțiunile corecte.

Pe lângă componenta practică și imediat utilă, această motivație are și o dimensiune mar-

cant mai profundă de formare și dezvoltare profesională personală. Dezvoltarea acestei aplicații a reprezentat un cadru ideal și un pretext potrivit pentru a aprofunda și a consolida cunoștințele în tehnologiile moderne de frontend web, în special în framework-ul Angular și ecosistemul lui, și pentru a aplica și testa în practică principii fundamentale de arhitectură software și design în context unui proiect cuprinzător și real, nu doar într-un prototip izolat sau într-un exercițiu academic. În sens mai larg, lucrarea combină oportun nevoia reală și tangibilă din mediul universitar cu aspirația personală de a construi o soluție care este robust proiectată, extensibilă și ușor de menținut și de evoluat în viitor.

Introducere

Digitalizarea administrației academice în instituțiile de învățământ superior nu se reduce la un simplu transfer al formularelor tradiționale într-un mediu electronic, ci presupune o re-gândire și o reproiectare fundamentală a fluxurilor de lucru, a modului în care informația circulă între actorii implicați, și a felului în care operațiile repetitive pot fi reduse la minimum prin automatizare inteligentă și prin design chibzuit. În forma lor clasică și inerțială, procesele administrative precum înscrierea și organizarea studenților în grupe de lucru, pregătirea și programarea sesiunilor de laborator, notarea și răspunderea prezenței participanților, sau centralizarea și reportarea situațiilor academice, au fost tradițional distribuite pe mai multe canale și instrumente disparate. Atunci când lipsește un sistem unificat care să integreze aceste procese și să le asigure o bază comună de date, consistența și corecția informațiilor depinde în mare măsură de disciplină și de buna pregătire a indivizilor, mai degrabă decât de regulile tehnice impuse de o platformă standardizată și automatizată.

Lucrarea de față propune proiectarea și implementarea tehnică a unei aplicații web dedicate satisfacerii acestor nevoi concrete și operaționale, o arhitectură client-server în care componentele sunt bine delimitată și responsabilități sunt clar definite. Frontend-ul, interfața cu care utilizatorul interacționează zilnic, este realizat cu ajutorul framework-ului Angular, o tehnologie modern de construcție a interfețelor web interactive și responsive. Backend-ul, stratul în care rezidă logica de business și deciziile critice ale aplicației, expune servicii prin intermediul unui API RESTful implementat în limbajul PHP, care este bine cunoscut, ușor de desfășurat, și larg suportat în infrastructura academică. Persistența datelor—stocarea și integritatea pe termen lung a informațiilor—este asigurată prin intermediul unei baze de date relaționale MySQL, tehnologie stabilă, scalabilă, și cu suport solid pe platformele LAMP tradiționale. Alegerea acestei arhitecturi, cu separarea clară între frontend, backend, și baza de date, a fost motivată de dorința de a realiza o separație coerentă și respecabilă a responsabilităților: interfața utilizatorului și experiența acestuia sunt gestionate exclusiv de frontend, logica complexă de business și validările critice sunt centralizate în backend, iar integritatea structurală și consistența semantică a datelor sunt garantate de stratul de bază de date.

Un accent deosebit al acestei lucrări este pus pe aspectele de frontend și pe framework-ul Angular, deoarece în această componentă se află stocate și se materializează majoritatea deciziilor de arhitectură care influențează direct și imediat experiența utilizatorului și mentenabilitatea pe termen lung a soluției. Sunt discutate și analizate teme importante: structurarea rațională a componentelor și modularitatea ierarhică, organizarea logică a modulelor funcționale, mecanismele de routare și protecția rutelor de acces, comunicarea între componente prin intermediul serviciilor și injecția de dependențe, lucrul reactiv și declarativ cu strumele de observabile RxJS, validarea avansată și reactivă a formularelor prin Reactive Forms, și integrarea ușoară a comunicării HTTP cu interceptori pentru gestionarea unificată a autentificării și a erorilor. Aceste elemente nu sunt prezentate abstract, ca o simplă enumerație de concepte sau de caracteristici, ci sunt analizate și exemplificate ca în piese care se legă armonic în cadrul unui flux funcțional complet și coerent.

Din punct de vedere funcțional și al cazurilor de utilizare, aplicația realizată acoperă setul de procese considerate esențiale pentru operarea unei facultăți sau a unui departament: mecanismele de autentificare și autorizare pe bază de roluri diferențiate, administrare și management al studenților și al grupelor lor de lucru, gestionarea cursurilor și a sesiunilor de laborator asociate, marcarea și urmărirea prezenței pe sesiuni, importul și actualizarea în masă a datelor din fișiere Excel, și vizualizarea de rapoarte și statistici sintetice. Fiecare modul funcțional al aplicației este analizat atât din perspective tehnice și de implementare, cât și din perspectiva utilizării practice și a impactului asupra experienței utilizatorului, pentru a evidenția și explica felul în care alegerile tehnice de implementare influențează viteza și fluiditatea operării, acuratețea și consistența datelor, și claritatea și intuitivitatea interfeței.

Din perspective metodologică și de proces de dezvoltare, realizarea acestei aplicații a urmat

un traseu ponderat și incremental, care a inclus etape succesive: definirea riguroasă a cerințelor și a constraintelor, modelarea riguros a datelor și a relațiilor dintre entități, proiectarea și documentarea arhitecturii sistemului, implementarea progresivă a modulelor și a componentelor, validarea soluției prin scenarii reale de utilizare și prin feedback-ul utilizatorilor pilot, și iterativ ajustări și îmbunătățiri pe baza observațiilor și a learnings-urilor. Această abordare incrementală și iterativă a permis rafinarea continuă și evoluția incrementală a soluției, menținând un echilibru dinamic între cerințele și constrângerile technice, pe de o parte, și utilitatea practică și relevența reală a soluției în contextul concret universitar, pe de altă parte.

Structura lucrării este organizată în patru capitole principale, fiecare servind un scop bine definit în construirea unei înțelegeri a temei. Capitolul întâi prezintă și disecă tehnologiile și framework-urile utilizate în dezvoltarea acestei aplicații, cu un accent extins și detaliat pe framework-ul Angular și pe mecanismele sale arhitecturale care sunt relevante și aplicabile în contextul acestui proiect. Capitolul al doilea descrie pe îndelete arhitectura sistemului, modelul relațional al datelor, și furnizează diagramele și explicațiile care clarific relațiile și fluxurile de date dintre componentele sistemului. Capitolul al treilea oferă o prezentare narativă și practică a aplicației prin fluxuri concrete de utilizare reale, de la momentul inițial al autentificării utilizatorului, trecând prin importul de date din fișiere Excel, până la procesele de marcare a prezenței și acces la rapoarte. Capitolul al patrulea, cel final, sintetizează rezultatele efectiv obținute prin implementare, identifică limitările și forțele sistemului, și propune direcții viabile de dezvoltare și îmbunătățire viitoare. Prin această structură coerentă și progresivă, lucrarea nu doar demonstrează că aplicația este funcțională și operațională, ci explică și justifică de ce soluția construită este potrivită și bine adaptată la problema adresată, cum poate fi extinsă și adaptată la contexte și cerințe în evoluție, și în ce mod aplicația poate sustine efectiv și durabil procesele academice în contexte reale pe termen lung.

Bibliografie

[1] Amazon Web Services, "What is a RESTful API?" https://aws.amazon.com/what-is/restful-api/, 2024, rEST architectural style originally defined by Roy T. Fielding in his 2000 doctoral dissertation.

[2] Git, "Git documentation," https://git-scm.com/doc, 2026, git created by Linus Torvalds in 2005; maintained by Junio C. Hamano since 2005.

[3] Google, "Angular v19 overview," https://v19.angular.dev/overview, 2024, angular originally created by Misko Hevery and Adam Abrons; AngularJS in 2010, modern Angular led by Igor Minar and the Angular team at Google.

[4] ——, "Angular CLI documentation," https://angular.dev/cli, 2026, developed and maintained by the Angular team at Google.

[5] JSON.org, "Introducing JSON," https://www.json.org/, 2026, jSON format specified and popularized by Douglas Crockford.

[6] JWT.io, "JSON web token (JWT) introduction," https://jwt.io/introduction/, 2026, jWT standardized in RFC 7519 by M. Jones, J. Bradley, and N. Sakimura (2015).

[7] MDN Web Docs, "HTTP: Hypertext transfer protocol," https://developer.mozilla.org/en-US/docs/Web/HTTP, 2026, hTTP/1.1 specified by Roy T. Fielding et al. (RFC 2616); HTTP/2 by M. Belshe, R. Peon, and M. Thomson.

[8] Oracle, "MySQL documentation," https://dev.mysql.com/doc/, 2024, mySQL originally created by Michael Widenius (Monty) and David Axmark in 1995; acquired by Oracle in 2010.

[9] PHP Group, "PHP manual," https://www.php.net/manual/, 2024, pHP created by Rasmus Lerdorf in 1994; PHP-FI and later versions developed with Andi Gutmans and Zeev Suraski.

[10] RxJS Team, "RxJS documentation," https://rxjs.dev/, 2024, reactive Extensions concept by Erik Meijer; RxJS port initiated by Matthew Podwysocki and the ReactiveX community.

[11] Stack Overflow, "Stack overflow developer survey 2025," https://survey.stackoverflow.co/2025/, 2025, stack Overflow co-founded by Jeff Atwood and Joel Spolsky in 2008.

[12] M. Takada, "Single page application book," https://singlepageappbook.com/, 2024.

[13] Wikipedia contributors, "Entity–relationship model," https://en.wikipedia.org/wiki/Entity%E2%80%93relationship_model, 2026, eR model originally proposed by Peter Chen in his 1976 paper.

[14] ScienceDirect, "Client-server architecture," https://www.sciencedirect.com/topics/computer-science/client-server-architecture, 2026, client-server model concept attributed to Xerox PARC researchers in the 1970s, notably Bob Metcalfe.